

nano-gpt

A Character-Level GPT (Transformer) From Scratch

A faithful PyTorch implementation of
Andrej Karpathy's "Let's build GPT" lecture.
Self-attention, multi-head, residual blocks and
every part of a decoder-only language model.

PyTorch

Transformer

Self-Attention

Char-level

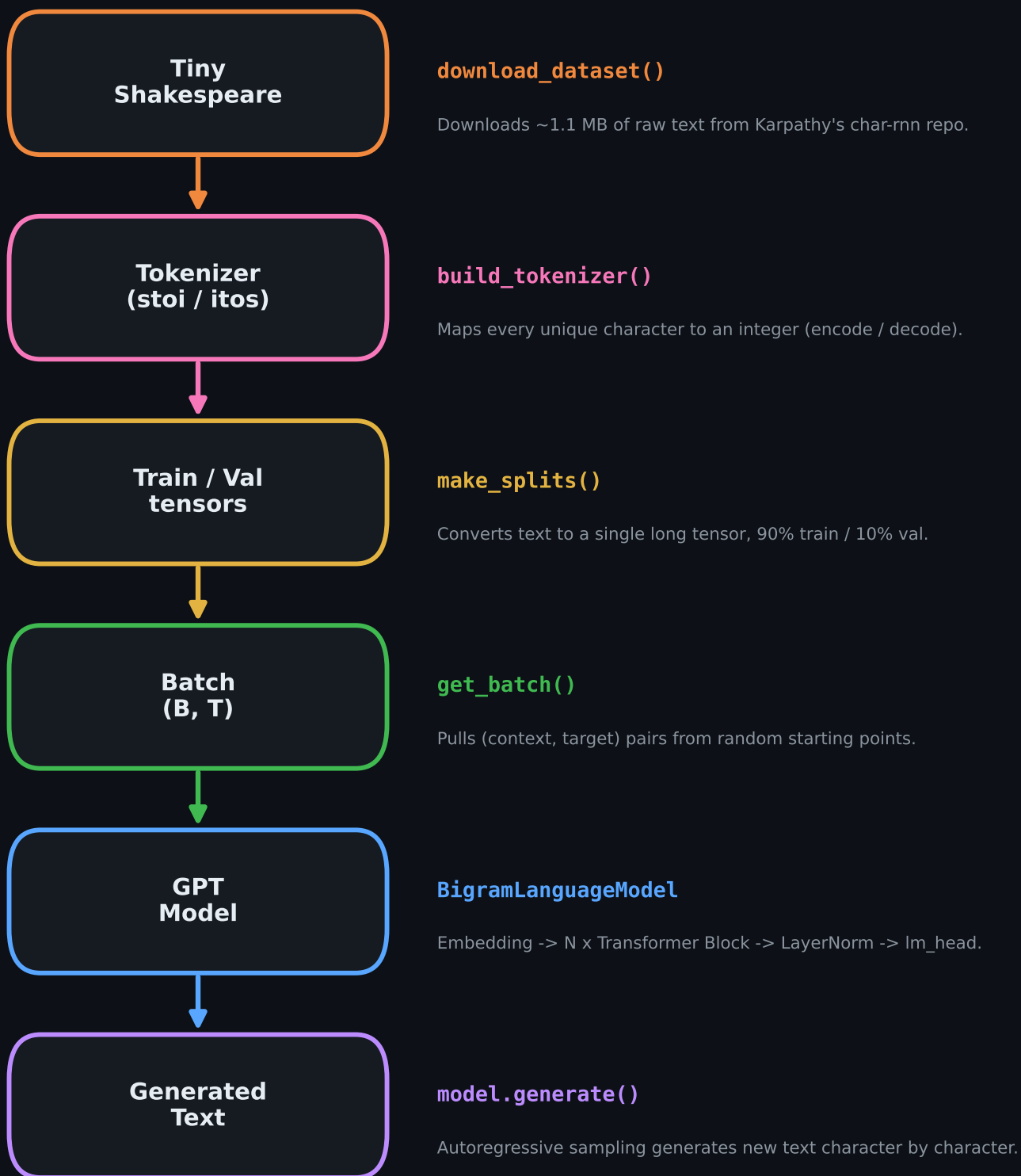
Weights & Biases

Architecture at a Glance

Dataset	Tiny Shakespeare (~1.1 MB text)
Tokenizer	Character-level · vocab $\approx$ 65
Model	Decoder-only Transformer · 6 blocks · 8 heads
Embedding size	$n_{\text{embed}} = 256$ · $\text{block_size} = 256$
Training	AdamW + Cosine LR · Early Stopping · W&B

End-to-End Data Flow

The full pipeline, from raw text to freshly generated text. Each box maps to a function or module in the notebook.



Tokenizer and Batch Construction

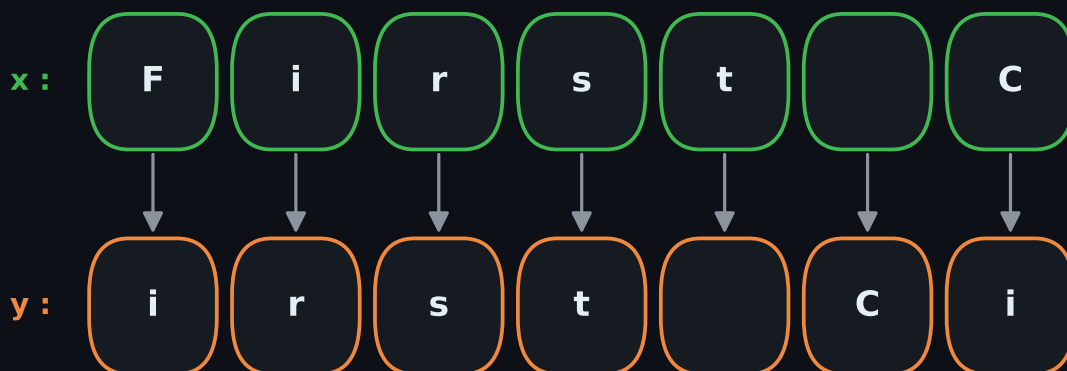
1 · Character-Level Tokenizer

Instead of BPE, each character is a single id. The vocab is tiny (~65), the embedding table is small, debugging is easy.



2 · `get_batch` → (context, target) pairs

A `block_size`-long `x` slice is paired with `y`, shifted one to the right. A single slice carries `T` independent training examples — this is the source of the Transformer's parallel learning.

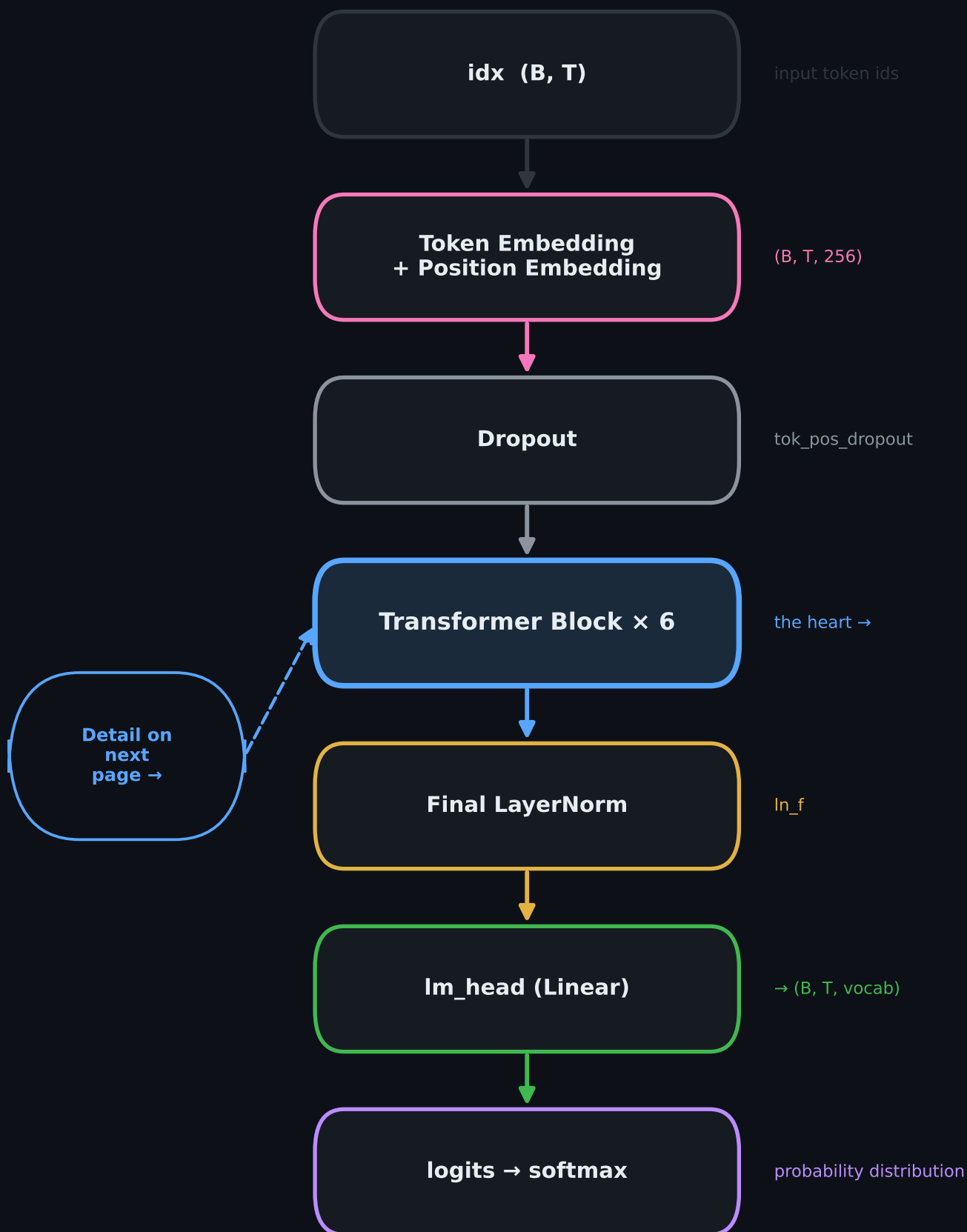


prediction direction: given $x[:, t]$ the model predicts $y[:, t]$

Tensor Shapes

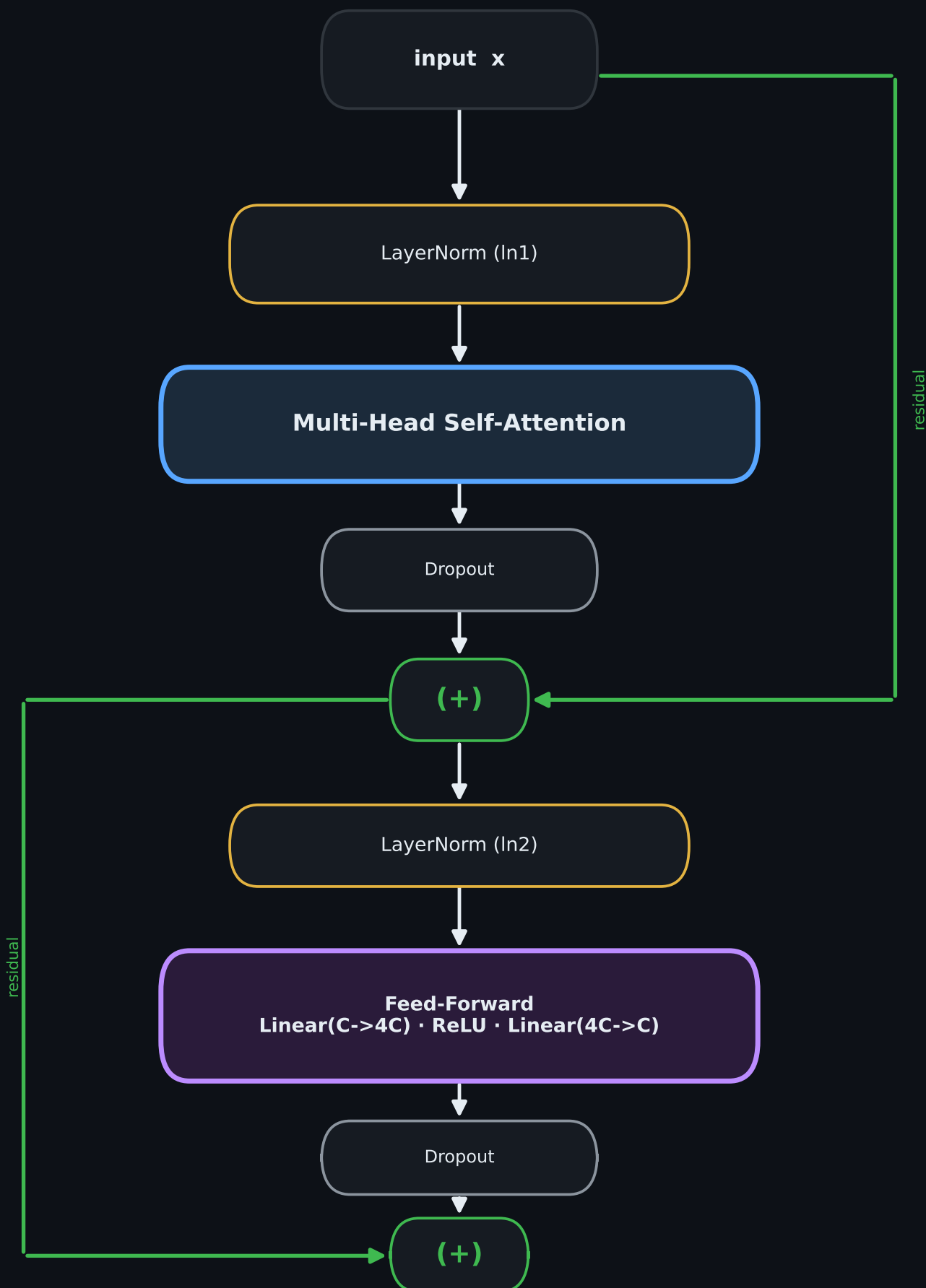
<code>x, y</code>	<code>(B, T)</code>	<code>B = batch_size, T = block_size</code>
<code>token_embedding</code>	<code>(B, T, C)</code>	<code>C = n_embed</code>
<code>logits</code>	<code>(B, T, vocab_size)</code>	score for each position
<code>loss</code>	scalar	cross-entropy

BigramLanguageModel — High-Level Flow



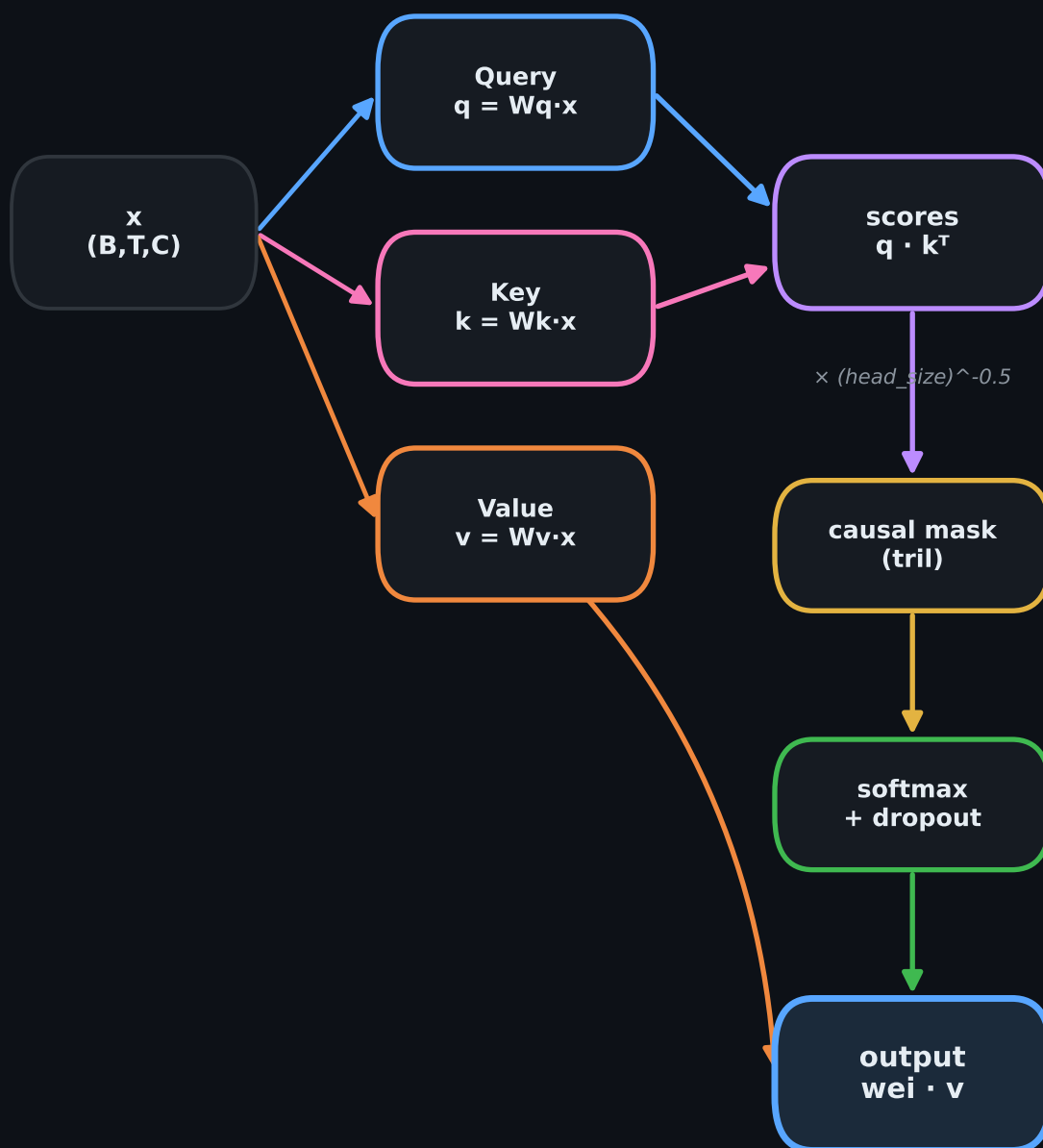
Transformer Block — Communicate + Compute

Each block has two sub-layers. Pre-LayerNorm is applied and the output of each sub-layer is added back to the input via a residual connection: $x = x + \text{Dropout}(\text{SubLayer}(\text{LN}(x)))$.



A Single Self-Attention Head

Each head splits the input into three projections: Query, Key, Value. Scores are scaled, the future is hidden with a causal mask, softmax turns them into weights, Values are summed.



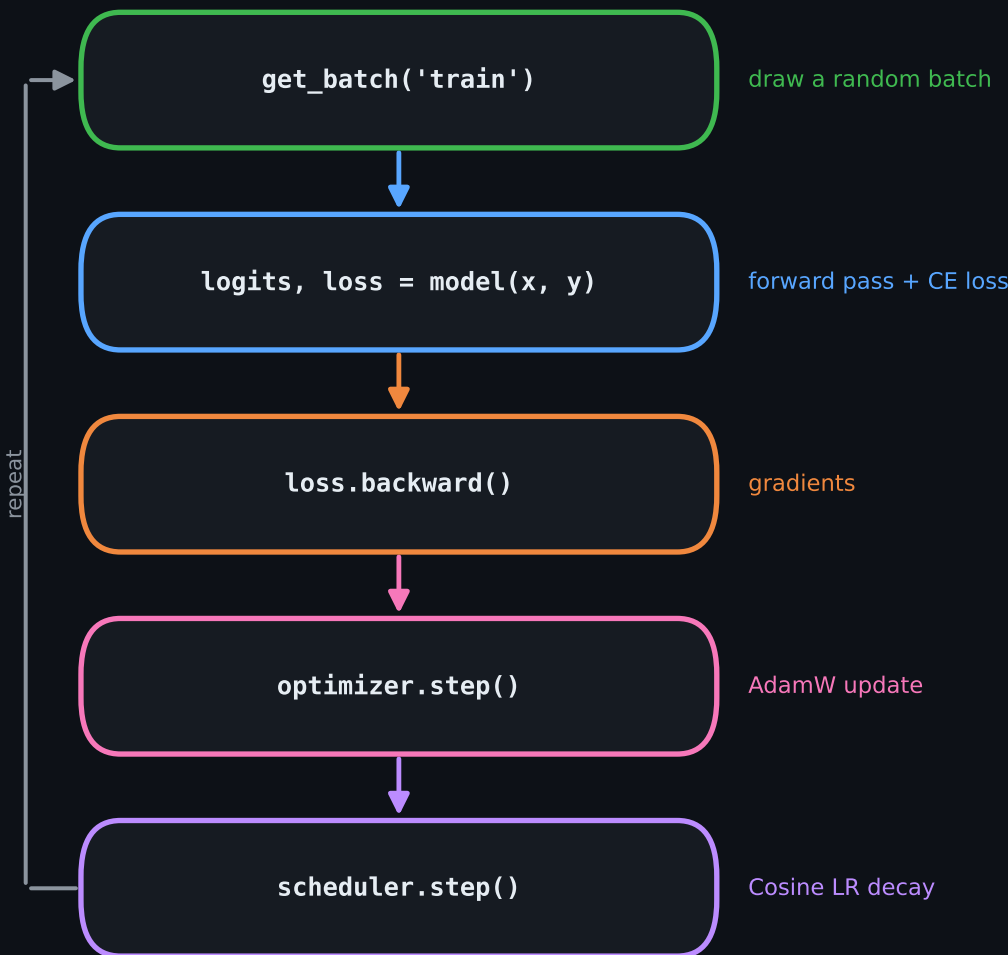
Causal mask: a token may only see itself and the past (future = -inf)

1	0	0	0	0
1	1	0	0	0
1	1	1	0	0
1	1	1	1	0
1	1	1	1	1

row = querying position
col = attended position

green (1) = allowed
dark (0) = masked

Iteration Loop



Regularization & Monitoring

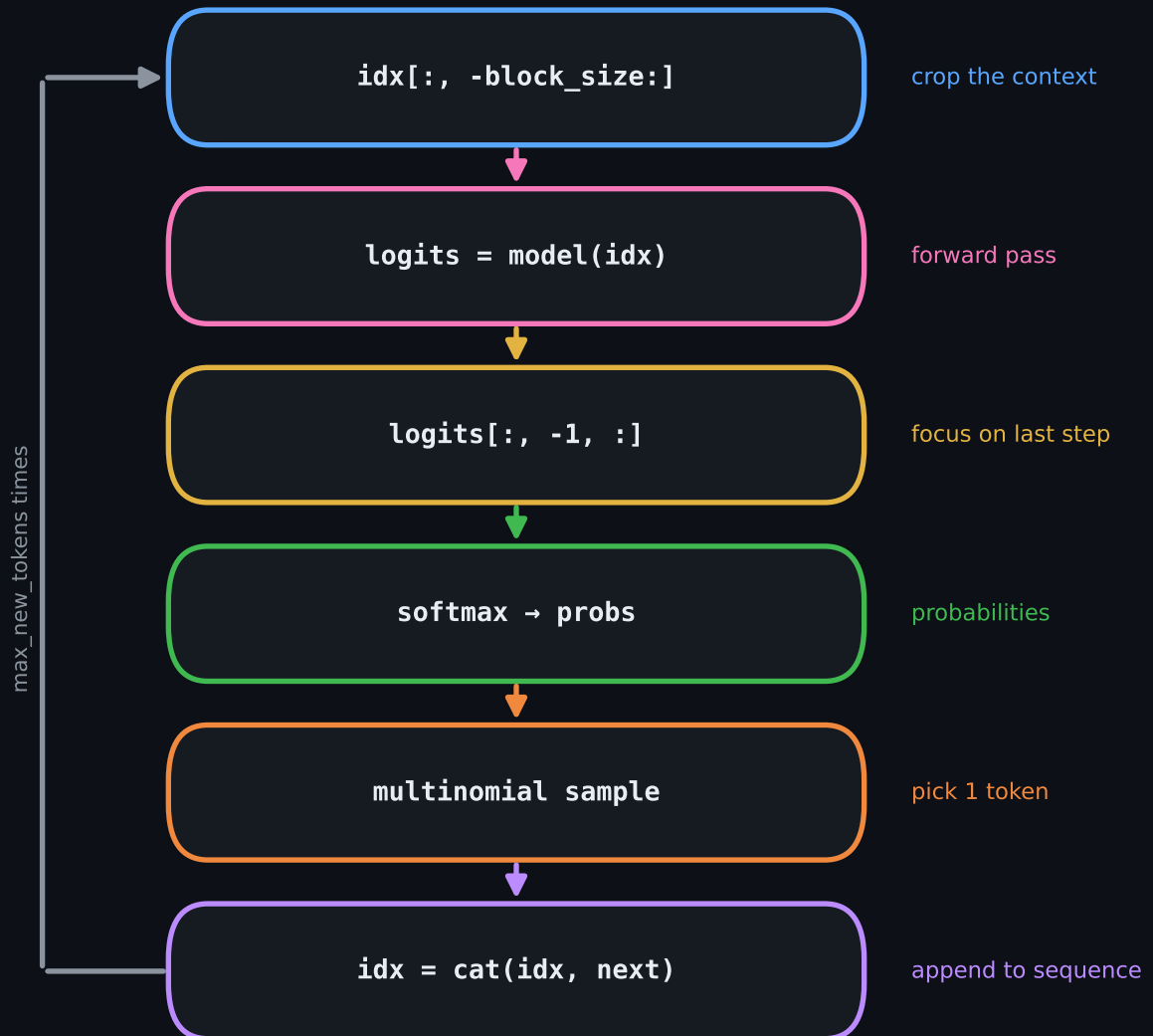
- **estimate_loss** estimates train/val loss every 500 steps (eval mode)
- **Early Stopping** stops if val loss stalls for 5 evaluations
- **Cosine LR** lr 5e-4 -> 1e-5, smooth decay
- **Weights & Biases** all metrics logged live

HERO RUN v4 — Configuration

block_size	256	batch_size	64	n_embed	256
n_head	8	n_layer	6	dropout	0.2
lr	5e-4	epochs	10000	weight_decay	1e-3

Autoregressive Text Generation

`model.generate()` takes the last `block_size` tokens as context at each step, samples one token from the last position's probability distribution and appends it. The loop feeds itself.



Why It Matters

This tiny model contains the exact same core mechanism as modern large language models (GPT): token + position embedding, causal self-attention, multi-head, residual blocks and autoregressive generation. Apart from scale, the fundamental architecture is identical — which makes it an ideal way to learn "GPT from scratch".

Results & the Bigram Limit

The notebook ships without stored run outputs, so the numbers below are reference values from Karpathy's lecture for the same char-level Tiny Shakespeare setup — not this run's logs.

Validation Loss (cross-entropy, lower is better)

Random init 4.17

= $\ln(65)$, pure chance over 65 chars

Pure bigram 2.50

1-char context only -> hits a floor

This Transformer 1.50

256-char context via attention

Reference: the scaled char-level GPT in the lecture reaches val loss ≈ 1.48 .

Why the Bigram Stalls

- **1-token context**
next char from the previous one only
- **No order info**
position is ignored entirely
- **Lookup table**
vocab $\times$ vocab, no composition
- **Loss floor**
plateaus around 2.5
- **Letter-soup**
output never forms real words

How the Transformer Fixes It

- ✓ **Self-attention**
context up to block_size (256)
- ✓ **Position embed.**
order now matters
- ✓ **Multi-head**
many relation types at once
- ✓ **Residual blocks**
depth + non-linear compute
- ✓ **Result**
loss ≈ 1.5 , Shakespeare-like text

Note on the name

The class is called `BigramLanguageModel` — a leftover from the lecture's starting point — but it is a full decoder-only Transformer. The "bigram" above is the bare-lookup baseline it evolved beyond.